

Value function Approximation Introduction

Master Degree in Control Systems Engineering 2024-2025
University of Padova

Teachers: Prof. Ruggero Carli & Prof. Gian Antonio Susto

Today's lecture and next lectures

- Introduction to the problem
- Incremental methods
- Batch methods

Today's lecture...

- Introduction to the problem
- Incremental methods
- Batch methods

Large-scale Reinforcement Learning

We would like to use reinforcement learning to solve *large* problems
(*non trivial state spaces that might be combinatorial and enormous*)

- Backgammon: 10^{20} states
- Computer Go: 10^{170} states
- Helicopter: continuous state space

In such cases we can not expect to find an optimal policy or the optimal value function

The goal is to find a **good approximation solution** using limited computational resources

Large-scale Reinforcement Learning

We would like to use reinforcement learning to solve *large* problems
(*non trivial state spaces that might be combinatorial and enormous*)

- Backgammon: 10^{20} states
- Computer Go: 10^{170} states
- Helicopter: continuous state space

In such cases we can not expect to find an optimal policy or the optimal value function

The goal is to find a **good approximation solution** using limited computational resources

How can we scale up the model-free methods for *prediction* and *control* from the last lectures?

Large-scale Reinforcement Learning

- So far we have represented value function by a *lookup table*
 - Every state s has an entry $V(s)$
 - Or every state-action pair s, a has an entry $Q(s, a)$

Large-scale Reinforcement Learning

- So far we have represented value function by a *lookup table*
 - Every state s has an entry $V(s)$
 - Or every state-action pair s, a has an entry $Q(s, a)$
- Problem with large MDPs:
 - There are too many states and/or actions to store in memory
 - It is too slow to learn the value of each state individually

Large-scale Reinforcement Learning

- So far we have represented value function by a *lookup table*
 - Every state s has an entry $V(s)$
 - Or every state-action pair s, a has an entry $Q(s, a)$
- Problem with large MDPs:
 - There are **too many states and/or actions** to store in memory
 - It is too slow to learn the value of each state individually

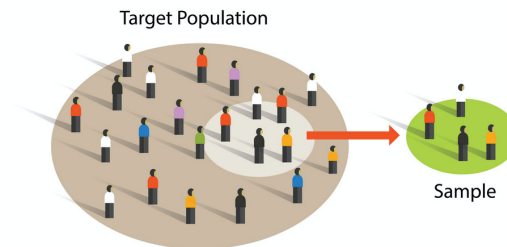
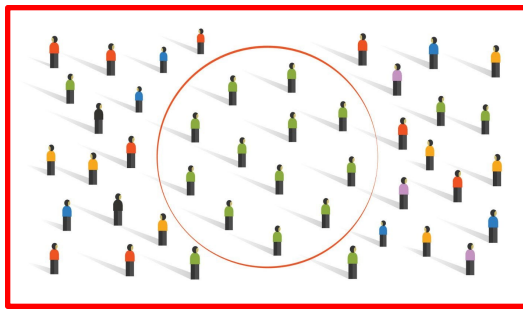
In many of the target tasks, *almost every state encountered will never have been seen before*

Large-scale Reinforcement Learning

- So far we have represented value function by a *lookup table*
 - Every state s has an entry $V(s)$
 - Or every state-action pair s, a has an entry $Q(s, a)$
- Problem with large MDPs:
 - There are **too many states and/or actions** to store in memory
 - It is too slow to learn the value of each state individually

In many of the target tasks, *almost every state encountered will never have been seen before*

...in such states it is necessary to **generalize** from previous encounters with different states which are in some sense similar to the current one



Generalization

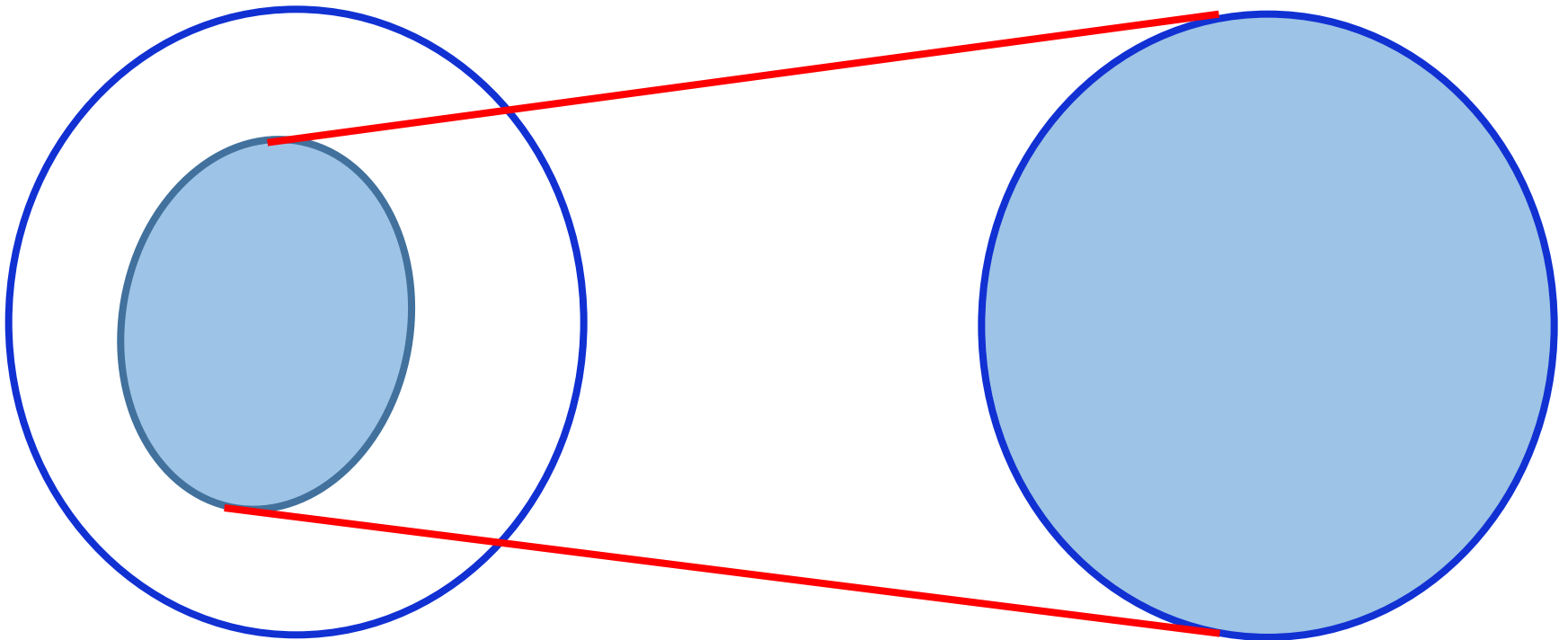
... *generalization*...

How can experience with a limited subset of the state space be usefully generalized to produce a good approximation over a much larger subset?

Generalization

... *generalization*...

How can experience with a limited subset of the state space be usefully generalized to produce a good approximation over a much larger subset?



Generalization

... *generalization*...

How can experience with a limited subset of the state space be usefully generalized to produce a good approximation over a much larger subset?

Function approximation (instance of supervised learning)



It takes *examples* from a
desired function
(e.g. a value function)

*attempts to
generalize*

...to construct an
approximation of the
entire function

More on Value function approx. – Supervised Learning

All of the prediction methods covered in this class have been described as *updates to an estimated value function that shift its value at particular states toward a “update target”*

More on Value function approx. – Supervised Learning

All of the prediction methods covered in this class have been described as updates to an estimated value function that shift its value at particular states toward a “update target”

Let us adopt the notation $s \mapsto u$

s : state updated

u : update target (that s 's estimated value is shifted toward)

Monte – Carlo update : $S_t \mapsto G_t$

TD(0) update : $S_t \mapsto R_{t+1} + \gamma V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha(u - V(S_t))$$

n-step update : $S_t \mapsto G_{t:t+n}$

$s \rightarrow u$ means that the estimated value for state s should be more like the update target u

More on Value function approx. – Supervised Learning

$s \rightarrow u$ can be seen as an example of the *desired input–output behavior* of the value function

More on Value function approx. – Supervised Learning

$s \rightarrow u$ can be seen as an example of the *desired input–output behavior* of the value function

So far, the actual update has been trivial: *the table entry for s 's estimated value has simply been shifted a fraction of the way toward u , and the estimated values of **all other states were left unchanged**.*

$$V(S_t) \leftarrow V(S_t) + \alpha(u - V(S_t))$$

More on Value function approx. – Supervised Learning

$s \rightarrow u$ can be seen as an example of the *desired input–output behavior* of the value function

So far, the actual update has been trivial: *the table entry for s 's estimated value has simply been shifted a fraction of the way toward u , and the estimated values of **all other states were left unchanged**.*

Now we permit arbitrarily complex and sophisticated methods to implement the update, and *updating at s generalizes so that the estimated values of many other states are changed* as well.

More on Value function approx. – Supervised Learning

Function approximation methods for value prediction : *artificial neural networks, decision trees, various kinds of multivariate regression, etc.*

However, not all function approximation methods are equally well suited for use in reinforcement learning (...most sophisticated neural network assume a **static training set** over which multiple passes are made)

More on Value function approx. – Supervised Learning

Function approximation methods for value prediction : *artificial neural networks, decision trees, various kinds of multivariate regression, etc.*

However, not all function approximation methods are equally well suited for use in reinforcement learning (...most sophisticated neural network assume a **static training set** over which multiple passes are made)

In reinforcement learning **learning occurs on-line**...

 ***Need of methods that are able to learn efficiently from incrementally acquired data***

Value function approximation

- Solution for large MDPs
 - Estimate value function with *function approximation*
$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$
or
$$\hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$$
 - *Generalise* from seen states to unseen states
 - *Update* parameter $\mathbf{w}$ using MC or TD learning

Value function approximation

- Solution for large MDPs
 - Estimate value function with *function approximation*
$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$
or
$$\hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$$
 - *Generalise* from seen states to unseen states
 - *Update* parameter $\mathbf{w}$ using MC or TD learning

As example, $\hat{v}$ might be the function computed by a multi-layer artificial neural network, with $\mathbf{w}$ the vector of connection weights in all the layers

Value function approximation

- Solution for large MDPs
 - Estimate value function with *function approximation*
$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$
or
$$\hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$$
 - *Generalise* from seen states to unseen states
 - *Update* parameter $\mathbf{w}$ using MC or TD learning

As example, $\hat{v}$ might be the function computed by a multi-layer artificial neural network, with $\mathbf{w}$ the vector of connection weights in all the layers

Typically the dimensionality of $\mathbf{w}$ (denoted as d or n) is much smaller than the number of states

Value function approximation

- Solution for large MDPs
 - Estimate value function with *function approximation*
$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$
or
$$\hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$$
 - *Generalise* from seen states to unseen states
 - *Update* parameter $\mathbf{w}$ using MC or TD learning

As example, $\hat{v}$ might be the function computed by a multi-layer artificial neural network, with $\mathbf{w}$ the vector of connection weights in all the layers

Typically the dimensionality of $\mathbf{w}$ (denoted as d or n) is much smaller than the number of states

- changing one weight changes the estimated value of many states

Value function approximation

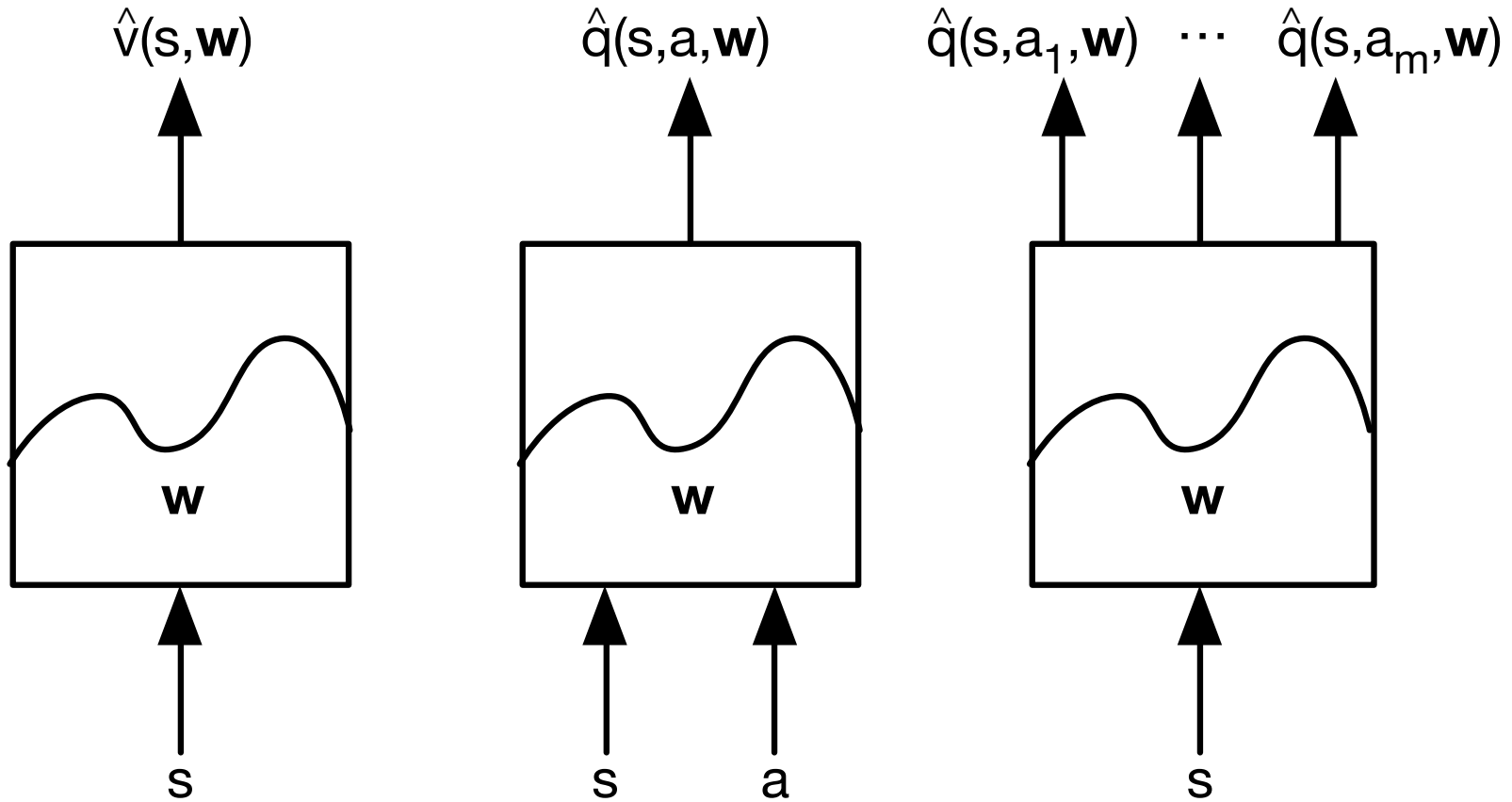
- Solution for large MDPs
 - Estimate value function with *function approximation*
$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$
or
$$\hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$$
 - *Generalise* from seen states to unseen states
 - *Update* parameter $\mathbf{w}$ using MC or TD learning

As example, $\hat{v}$ might be the function computed by a multi-layer artificial neural network, with $\mathbf{w}$ the vector of connection weights in all the layers

Typically the dimensionality of $\mathbf{w}$ (denoted as d or n) is much smaller than the number of states

- changing one weight changes the estimated value of many states
- when a single state is updated, the change generalizes from that state to affect the values of many other states

Type of value function approximations



Which function approximator

We consider **differentiable** function approximators, e.g.

- **linear combinations of features**
- **Neural network**
- Decision tree
- Nearest neighbour
- Fourier/ wavelet bases
- ...